Insertion in Linked List

A **Linked List** is a linear data structure where elements are stored in **nodes**.

Each node contains:

1. **Data**
2. **Pointer/Reference** to the next node

Types of Insertion

1. Insert at Beginning
2. Insert at End

---

1. Insertion at Beginning

Intuition

Suppose list is:

10 → 20 → 30

Now insert 5 at beginning.

Steps:

1. Create new node 5
2. Point new node to current head
3. Move head to new node

Result:

5 → 10 → 20 → 30

Visualization

Before:

head
↓
10 → 20 → 30 → NULL

Create new node:

5 → NULL

Connect new node to head:

5 → 10 → 20 → 30 → NULL

Move head:

head
↓
5 → 10 → 20 → 30 → NULL

Algorithm

1. Create new node
2. Store data
3. newNode.next = head
4. head = newNode

Time Complexity

Operation **Complexity**

Insert at Beginning O(1)

Because no traversal needed.

Python Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_beginning(self, data):
        newNode = Node(data)

        newNode.next = self.head
        self.head = newNode

    def display(self):
        temp = self.head

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
        print("NULL")


ll = LinkedList()

ll.insert_beginning(30)
ll.insert_beginning(20)
ll.insert_beginning(10)

ll.display()
```

C++ Code

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

class LinkedList {
public:
    Node* head;

    LinkedList() {
        head = NULL;
    }

    // Insert at beginning
    void insertBeginning(int data) {
        Node* newNode = new Node(data);

        newNode->next = head;
        head = newNode;
    }

    void display() {
        Node* temp = head;
```

```
        while(temp != NULL) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }

        cout << "NULL" << endl;
    }
};

int main() {
    LinkedList ll;

    ll.insertBeginning(30);
    ll.insertBeginning(20);
    ll.insertBeginning(10);

    ll.display();

    return 0;
}
```

Java Code

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class LinkedList {
    Node head;

    // Insert at beginning
    void insertBeginning(int data) {
        Node newNode = new Node(data);

        newNode.next = head;
        head = newNode;
    }

    void display() {
        Node temp = head;
```

```
        while(temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }

        System.out.println("NULL");
    }

    public static void main(String[] args) {
        LinkedList ll = new LinkedList();

        ll.insertBeginning(30);
        ll.insertBeginning(20);
        ll.insertBeginning(10);

        ll.display();
    }
}
```

Interview Explanation

Why insertion at beginning is O(1)?

Because:

- We only change head pointer
- No traversal required

Only constant operations happen.

Common Mistake

Wrong:

```
head = newNode;
newNode.next = head;
```

Why wrong?
Because old head is lost.

Correct:

```
newNode.next = head;
head = newNode;
```

---

2. Insertion at End

Intuition

Suppose:

10 → 20 → 30

Insert 40 at end.

We must:

1. Create new node
2. Traverse till last node
3. Last node next = new node

Result:

10 → 20 → 30 → 40

Visualization

Before:

head
↓
10 → 20 → 30 → NULL

Traverse to last node:

30 → NULL

Connect:

30 → 40 → NULL

Algorithm

1. Create new node
2. If head is NULL:
    - head = newNode
3. Else:
    - Traverse till temp.next != NULL
    - temp.next = newNode

Time Complexity

Operation Complexity

Operation Complexity

Insert at End O(N)

Because traversal required.

Python Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # Insert at end
    def insert_end(self, data):
        newNode = Node(data)

        # Empty linked list
        if self.head is None:
            self.head = newNode
            return

        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = newNode

    def display(self):
        temp = self.head

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("NULL")


ll = LinkedList()

ll.insert_end(10)
```

```
ll.insert_end(20)
ll.insert_end(30)

ll.display()
```

C++ Code

```
#include <iostream>
using namespace std;

class Node {
public:
   int data;
   Node* next;

   Node(int val) {
      data = val;
      next = NULL;
   }
};

class LinkedList {
public:
   Node* head;

   LinkedList() {
      head = NULL;
   }

   // Insert at end
   void insertEnd(int data) {
      Node* newNode = new Node(data);

      // Empty list
      if(head == NULL) {
         head = newNode;
         return;
      }

      Node* temp = head;

      while(temp->next != NULL) {
         temp = temp->next;
      }

      temp->next = newNode;
   }
```

```
    void display() {
        Node* temp = head;

        while(temp != NULL) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }

        cout << "NULL" << endl;
    }
};

int main() {
    LinkedList ll;

    ll.insertEnd(10);
    ll.insertEnd(20);
    ll.insertEnd(30);

    ll.display();

    return 0;
}
```

Java Code

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class LinkedList {
    Node head;

    // Insert at end
    void insertEnd(int data) {
        Node newNode = new Node(data);

        // Empty list
        if(head == null) {
            head = newNode;
```

```
            return;
        }

        Node temp = head;

        while(temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }

    void display() {
        Node temp = head;

        while(temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }

        System.out.println("NULL");
    }

    public static void main(String[] args) {
        LinkedList ll = new LinkedList();

        ll.insertEnd(10);
        ll.insertEnd(20);
        ll.insertEnd(30);

        ll.display();
    }
}
```

Interview Notes

Why insertion at end is O(N)?

Because:

- We must traverse entire linked list
- Need to reach last node

Traversal takes linear time.

Optimization Using Tail Pointer

If we maintain:

head + tail

Then insertion at end becomes:

Operation	Complexity
Insert at End using Tail	O(1)

Because tail directly points to last node.

Edge Cases

1. Empty Linked List

head = NULL

Both beginning and end insertion should make:

head = newNode

---

2. Single Node List

10 → NULL

Insertion should work properly.

Difference Between Beginning and End Insertion

Feature	Beginning	End
Traversal Needed	No	Yes
Time Complexity	O(1)	O(N)
Pointer Changes	Head only	Last node next
Easier	Yes	Slightly harder

Important Interview Points

Insert at Beginning

newNode.next = head
head = newNode

Remember order carefully.

Insert at End

Traversal condition:

while(temp.next != null)

Why not:

while(temp != null)

Because:

- We need last node
- Not NULL position

Real World Usage

Insert at Beginning

Used in:

- Stack implementation
- Undo operations
- Browser history

Insert at End

Used in:

- Queue implementation
- Music playlist
- Chat messages